

Introduction to R

Session 02: Working with Data

Álvaro Pérez¹

Instituto Tecnológico Autónomo de México

Spring 2025

¹Based on PhD Romero Londoño's notes.

- ▶ R is all about working with data
- ▶ `data.frames`:
 - ▶ `data.frames` are an object type
 - ▶ Most of the time, you'll be doing calculations using them
 - ▶ Conceptually, `data.frames` are basically spreadsheets
 - ▶ Technically, they're a list of vectors

- It's a collection of vectors of the same length
- Note the use of = here, not <-

```
1 df <- data.frame(  
2   RacePosition = 1:5,  
3   WayTheySayHi = as.factor(c('Hi', 'Hello', 'Hey', 'Yo', 'Hi'  
4     ')),  
5   NumberofKids = c(3,5,1,0,2))  
df
```

- ▶ If we just want a quick overview:
 - ▶ Arrow in the Environment tab
 - ▶ 'head()' (look at the head of the data - first six rows)
 - ▶ 'str()' (structure)

1

```
str(df)
```

- ▶ Now we have a data frame, 'df'. How do we use it?
- ▶ We can pull the vectors back out with '\$' Note autocompletion of variable names.
- ▶ We can treat it just like the vectors we had before

```
1 df$NumberofKids  
2 df$NumberofKids[2]  
3 df$NumberofKids >= 3
```

- ▶ Type the name of a dataset into the console to see the data. Type "smalldata" into the console to see the dataset description.
- ▶ You can also use the **data()** function to load a dataset into your global environment.
- ▶ The function **glimpse()** gives a sideways version of the tibble. This is useful if the table is very wide and you can't see all of the columns. It also tells you the data type of each column in angled brackets after each column name.
- ▶ You can get a quick summary of a dataset with the **summary()** function.

Extension	File Type	Reading	Writing
.csv	Comma-separated values	<code>readr::read_csv()</code>	<code>readr::write_csv()</code>
.tsv, .txt	Tab-separated values	<code>readr::read_tsv()</code>	<code>readr::write_tsv()</code>
.xls, .xlsx	Excel workbook	<code>readxl::read_excel()</code>	NA
.sav, .mat, ...	Multiple types	<code>rio::import()</code>	NA

- ▶ Create `df2 <- data.frame(a = 1:20, b = 0:19*2, 'c' = sample(101:200,20,replace=TRUE))`
- ▶ What is the average of 'c'?
- ▶ What is the sum of 'a' times 'b'?
- ▶ Did you get any values of 'c' 103 or below? (make a logical)
- ▶ What is on the 8th row of 'b'?
- ▶ How many rows have 'b' above 10 AND 'c' below 150?


```
1 df2 <- data.frame(  
2   a = 1:20,  
3   b = 0:19*2,  
4   c = sample(101:200,20,replace=TRUE))  
5  
6 mean(df2$c)  
7  
8 sum(df2$a*df2$b)  
9  
10 sum(df2$c <= 103) > 0  
11  
12 df2$b[8]  
13  
14 sum(df2$b > 10 & df2$c < 150)
```

- ▶ We can manipulate data frames:
 - ▶ Create new variables
 - ▶ Change variables
 - ▶ Rename variables
- ▶ It's very common that you'll have to work with data before analyzing it
- ▶ “data cleaning” is very important and a big part of statistical analysis

- ▶ data.frames are just lists of vectors
- ▶ So create a vector and tell R where in that list to stick it
- ▶ Use descriptive names so you know what the variable is

```
1 df$State <- c('Alaska', 'California',  
2               'California', 'Maine',  
3               'Florida')  
4 df
```

- ▶ We just saw the base-R way to do it
- ▶ Can use **dplyr** (data pliers) for data manipulation instead
- ▶ dplyr syntax is inspired by SQL
- ▶ tidyverse isn't a part of base R. It's in a package, so we'll need to install it

Variable creation with dplyr

- ▶ The **mutate** command will “mutate” our data frame to have a new column in it
- ▶ The pipe ‘%>%’ says “take df and send it to that mutate command to use”
- ▶ Or we can stick the data frame itself in the ‘mutate’ command
- ▶ Thus these two are equivalent:

```
1 library(tidyverse)
2 df1 <- df %>% mutate(State = c('Alaska', 'California',
3                                'California', 'Maine', '
4                                Florida'))
5
6 df2 <- mutate(df, State = c('Alaska', 'California',
7                             'California', 'Maine', '
8                             Florida'))
9
10 identical(df1, df2)
11 df <- df1
```

- We can create multiple new variables in one mutate command

```
1 df <- df %>% mutate(  
2   MoreThanTwoKids = NumberofKids > 2,  
3   One = 1,  
4   KidsPlusPosition = NumberofKids + RacePosition)  
5  
6 df
```

- ▶ We can't really **change** variables, but we can overwrite them
- ▶ We can drop variables with '-' in the dplyr 'select' command
- ▶ Note we chain multiple dplyr commands with '%> %'

```
1 df <- df %>%  
2   select(-KidsPlusPosition, -WayTheySayHi, -One) %>%  
3   mutate(State = as.factor(State),  
4           RacePosition = RacePosition - 1)  
5 df$State[3] <- 'Alaska'  
6 str(df)
```

- Sometimes it will make sense to change the names of the variables we have.
- Names are stored in 'names(df)' which we can edit directly
- Or the 'rename()' command in dplyr has us covered

```
1 names(df)
2 #two ways of renaming
3 #names(df) <- c('Pos', 'Num.Kids', 'State', 'mt2Kids
4   ')
5 df <- df %>% rename(Pos = RacePosition, Num.Kids=
6   NumberofKids,
7   mt2Kids = MoreThanTwoKids)
8 names(df)
```


- ▶ Create a data set 'data' with three variables: 'a' is all even numbers from 2 to 20, 'b' is 'c(0,1)' over and over, and 'c' is any ten-element numeric vector of your choice.
- ▶ Rename them to 'EvenNumbers', 'Treatment', 'Outcome'.
- ▶ Add a logical variable called Big that's true whenever EvenNumbers is greater than 15
- ▶ Increase Outcome by 1 for all the rows where Treatment is 1.
- ▶ Create a logical AboveMean that is true whenever Outcome is above the mean of Outcome.
- ▶ Display the data structure

```
1 data <- data.frame(a = 1:10*2,  
2                   b = c(0,1),  
3                   c = sample(1:100,10,replace=FALSE))  
4                   %>%  
5   rename(EvenNumbers = a, Treatment = b, Outcome = c)  
6  
7 data <- data %>%  
8   mutate(Big = EvenNumbers > 15,  
9          Outcome = Outcome + Treatment,  
10         AboveMean = Outcome > mean(Outcome))  
11  
12 str(data)
```

- ▶ `select()`: used to select columns of a data frame that you want to focus on
- ▶ `filter()`: used to extract subsets of rows from a data frame
- ▶ `arrange()`: used to reorder rows of a data frame according to one of the variables/columns
- ▶ `rename()`: renaming a variable
- ▶ `mutate()`: compute transformations of variables in a data frame
- ▶ `group_by()`: used to generate summary statistics from the data frame within strata defined by a variable.
- ▶ `%>%`: stringing together multiple dplyr functions in a sequence of operations.
- ▶ `summarize()`

- ▶ Download the database "1976-2020-president.csv" from my github [repository](#)
- ▶ In R, upload the dataframe
- ▶ Preserve only the votes for the Democratic and the Republican Party from de 1976 elections
- ▶ Create a new dataframe with only the state, the party, the votes for each party and the total of votes
- ▶ Sort states by number of votes
- ▶ Rename de variable "party_simplified", as "party"
- ▶ Create new variable that is the percentage of the vote (use the dataframe of the second step)
- ▶ Group by party and calculate de total aumount of votes for each party in that election

```

1 #install.packages("dplyr")
2 library(dplyr)
3 # Rename database
4 US_elections <- X1976_2020_president
5 head(US_elections)
6 str(US_elections)
7 # filter()
8 elect_1976 <- filter(US_elections, year == 1976 & (
    party_simplified == "DEMOCRAT" | party_simplified
    == "REPUBLICAN"))
9 # select()
10 votes_1976 <- select(elect_1976, state | party_
    simplified | candidatevotes)
11 # arrange()
12 arrange(votes_1976, candidatevotes)
13 # rename()
14 votes_1976 <- rename(votes_1976, "party"=party_
    simplified)

```

```
1
2 # mutate()
3 elect_1976 <- elect_1976 %>%
4   mutate(per_votes = candidatevotes/totalvotes)
5 # group_by()
6 votes_1976_party <- votes_1976 %>%
7   group_by(parties) %>%
8   summarise(sum_votes=sum(candidatevotes, na.rm = T))
```